



Akarsh Vyas

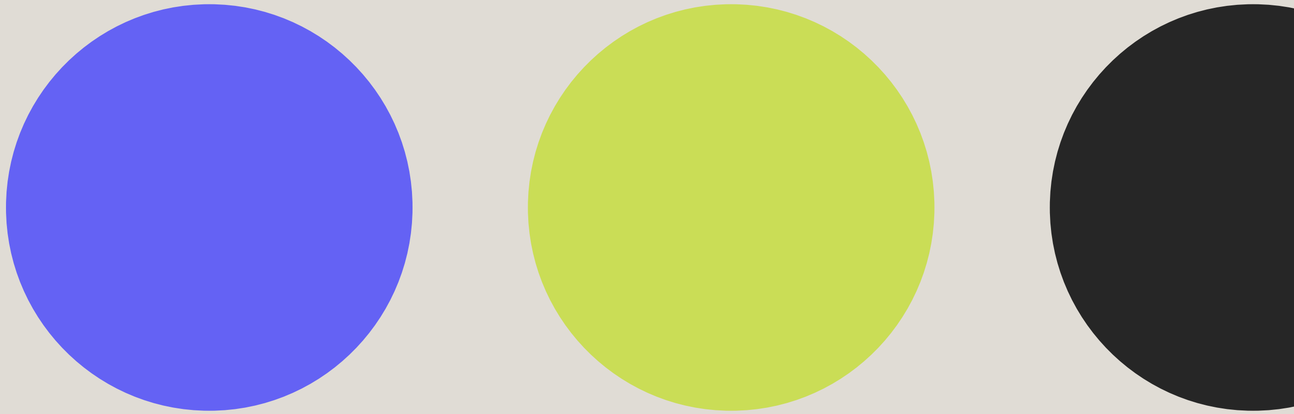
Generative AI

Prerequisites

Before starting there are things that you should know.

1. Python Programming
2. Machine Learning Fundamentals
3. Deep Learning Concepts
4. Understanding of Transformers
5. Basic NLP (tokenization, embeddings)

Here’s some important phases before starting.



Phase 1
Research layer

Phase 2
Bedrock models



Phase 3 Providers

Phase 4
This is where you comes in picture

You are going to use these LLM models and create some great AI applications.

Plan of Action

1. LLM foundation
2. RAG system
3. Agentic AI
4. project and deployment

Question 1
what is LLM ?

LLM = Large Language Model
Now break it into 3 words:

1. Large
It is trained on a huge amount of text data
(books, websites, Wikipedia, articles, code, conversations, etc.)

2. Language
It understands and generates human language
Like English, Hindi, Spanish... even programming languages like Python.

3. Model
It is a deep learning system (a neural network) that learns patterns from data.

Question 1
what is LLM ?

When you type:
“What is Python?”

The LLM does NOT:
- **Search Google**
- **Think like a human**
- **Understand meaning like we do**

Instead it:
- **Looks at patterns it learned**
- **Predicts the next most likely word**
- **Keeps predicting word by word**

Some popular LLM’s

GPT Models
Gemini Models
LLaMA
Claude
Grok
Mistral ... etc

You might have seen or heard about these large models.
In Generative AI, we don't build them from scratch
we learn how to use these pre-built LLMs effectively.

But how to use these models?

Open AI

```
from openai import OpenAI

# Create client
client = OpenAI()

# Send request to model
response = client.chat.completions.create(
    model="gpt-4o-mini",  -O
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Explain what is a Large Language Model in simple terms."}
    ],
)

# Print output
print(response.choices[0].message.content)
```

Gemini

```
import google.generativeai as genai
import os

# Configure API key
genai.configure(api_key=os.environ["GOOGLE_API_KEY"])

# Choose model
model = genai.GenerativeModel("gemini-1.5-flash")

# Send prompt
response = model.generate_content(
    "Explain what is a Large Language Model in simple terms."
)

# Print response
print(response.text)
```

claude

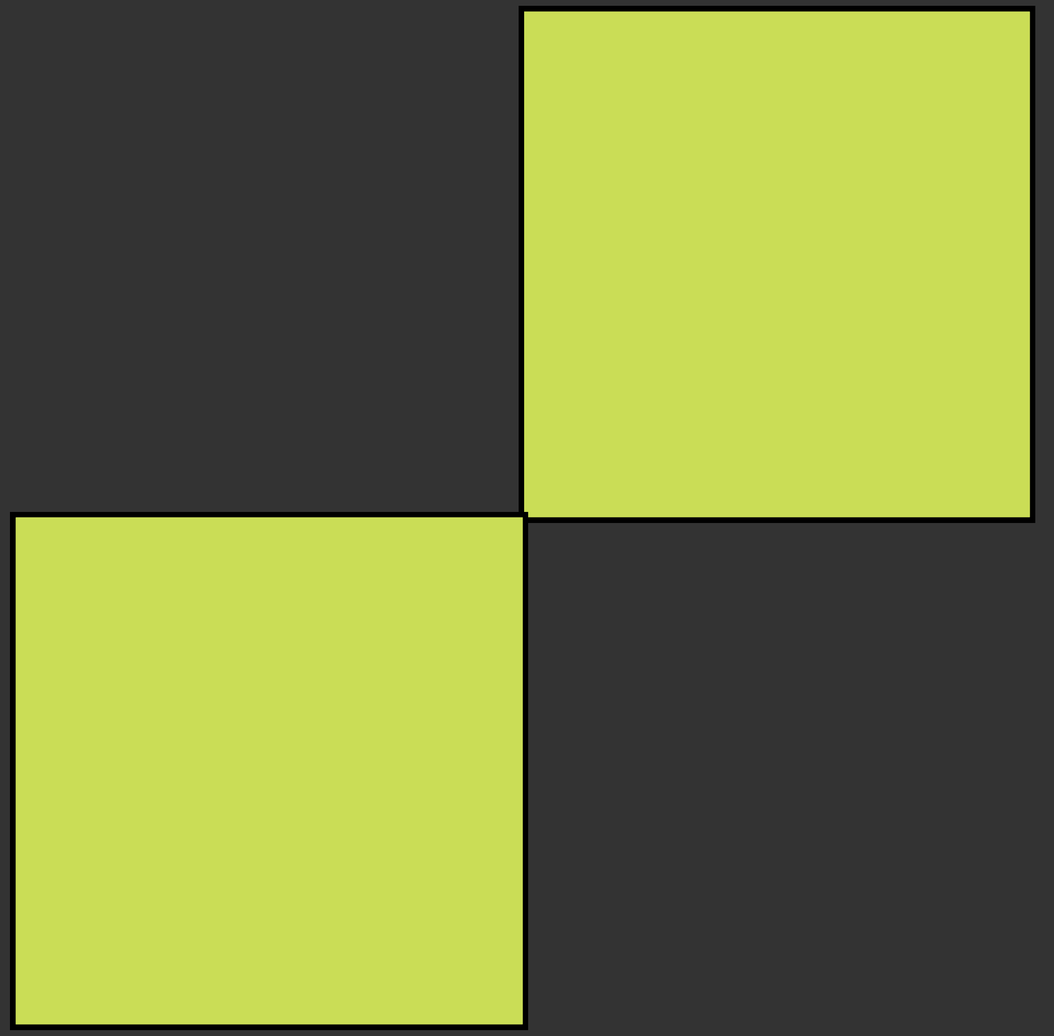
```
import anthropic
import os

client = anthropic.Anthropic(
    api_key=os.environ["ANTHROPIC_API_KEY"]
)

response = client.messages.create(
    model="claude-3-5-sonnet-latest",
    max_tokens=300,
    messages=[
        {"role": "user", "content": "Explain what is a Large Language Model in simple terms."}
    ],
)
print(response.content[0].text)
```

Problem

If every provider has a different SDK,
different syntax,
different response format...
How do we build scalable AI applications?



Langchain is the solution



Components

- Models
- Prompts
- Chains
- Memory
- Indexes
- Agents

Models

Models are the core intelligence layer.
They generate text, embeddings, or other outputs.
They connect your application to LLM providers like:

- OpenAI
- Google DeepMind
- Anthropic

Models

In practical Generative AI, there are mainly 3 important types of models:

Chat / Language Models

1. Generate text
2. Answer questions
3. Write code
4. Summarize content

Embedding Models

1. Convert text into numbers (vectors)
2. Used in search & RAG
3. Help find similar content

Multimodal Models

1. Work with more than just text
2. Can process images, audio, or files

Prompts

A prompt is the instruction we give to a language model.

It tells the model:

- What to do
  - How to respond
  - What format to use

Without prompts, the model has no direction.

Prompts

LLMs are powerful but they are not mind readers.
The quality of output depends on:
How clearly we instruct them.
Better prompt → Better output.

Prompts

Types of prompts

1. Simple Prompt

Just a direct instruction.
Example:
Explain what is Machine Learning.
Basic. Straightforward.

2. System + User Prompt

Modern chat models use roles:

- System → Sets behavior
- User → Asks question

Example:
System:
You are a helpful AI teacher.
User:
Explain LLMs in simple terms.
System controls tone and style.

Prompts

Types of prompts

3. Prompt Templates

Instead of writing prompts manually every time, we create reusable templates.
Example:
Explain {topic} in simple terms.
Now you can replace {topic} dynamically.
This is useful in applications.

4. Structured output

Used when we want:

- JSON output
- Bullet points
- Fixed format

Example:
Respond in JSON format with keys: definition, example.
This makes output machine-readable.

Chains

A Chain is a sequence of steps connected together.
Instead of:
One Prompt → One Response
We create:
Step 1 → Step 2 → Step 3 → Final Output

Chains

Real AI applications are not one-step.
Example:
User asks:
Summarize this article and translate it into Hindi.
What actually happens?

1. Summarize text
2. Translate summary
3. Return result

That’s a chain.

Memory

Memory allows the AI system to remember past interactions.
Without memory:
Every message is treated independently.
With memory:
The model can use previous context.

Memory

Example:
User: My name is Akarsh.
Later...
User: What is my name?

Without memory → Model doesn’t know.
With memory → It remembers.

Indexes

Indexes allow us to connect external data to an LLM.
LLMs only know what they were trained on.
Indexes help them use:

- PDFs
- Documents
- Databases
- Company data
- Websites

Indexes

LLMs:
❌ Don’t know your company data
❌ Don’t know private documents
❌ Don’t update in real-time

So how do we solve this?
We connect external knowledge.
That system is called Retrieval.

Agents

Agents are AI systems that can decide what action to take.
Unlike normal chains:
Chains → Follow fixed steps
Agents → Decide the next step dynamically

Agents

Normal LLM:
User → Prompt → Response
But what if the task requires:

- Searching the web
- Doing calculations
- Calling an API
- Using a database

The LLM alone cannot do that.
It needs tools.

Enough for now
Lets code



Welcome Back



Recap

- ⟶ LLM foundation
- ⟶ API keys
- ⟶ Used the Chat models
- ⟶ Used hugging face and local model
- ⟶ Used messages
- ⟶ Prompts
- ⟶ Structured output

Now you know me guys
we have to complete everything
about RAG in this video but while
teaching Generative AI I got to
know the best way of teaching
this is project based learning

So Lets start with a great project

CourseMate AI

CourseMate AI is an AI-powered study assistant designed to help students interact with their learning materials more efficiently. Modern students rely on multiple sources of study material such as lecture notes, textbooks, PDFs, and research papers. These documents are often long and difficult to navigate, making it time-consuming to find specific information.

CourseMate AI

CourseMate AI aims to solve this problem by allowing students to chat with their study materials. Instead of manually searching through pages of content, students can simply ask questions and receive accurate answers extracted directly from their documents.

CourseMate AI

By leveraging Retrieval-Augmented Generation (RAG), CourseMate AI combines document retrieval with large language models to provide context-aware explanations, summaries, and answers from the student's own study resources.

Development plan

Now to develop this kind of system we are going to follow some steps and the good thing is we are going to apply each and every step and create our project as well.

toh padhai bhi ho jaaye gi and project bhi ban jaaye ga.

Development plan

Step 1 - User Uploads Study Material

User Uploads Study Material
Students upload learning resources such as:

- PDFs
- lecture notes
- textbooks
- research papers

Development plan

Step 2 - Document Loading

The system loads documents using document loaders.
Goal:
Convert raw files into document objects that can be processed.

you may clean the document as well

Development plan

Step 3 - Text Splitting (Chunking)

Documents are usually too large for LLM context windows.
So we split them into smaller chunks.

Chunking improves retrieval accuracy.

Development plan

Step 4 - Embedding Generation

Each chunk is converted into a vector embedding.
Embedding models transform text into numerical vectors.

"Gradient Descent Optimization"
↓
[0.23, -0.81, 0.44, ...]

These vectors represent semantic meaning.

Development plan

Step 5 - Vector Database Storage

All embeddings are stored inside a vector database.
The vector database stores:

- embeddings
- original text chunks
- metadata

Development plan

Step 6 - User Asks a Question

Now the student interacts with the system.

Development plan

Step 7 - Query Embedding

The question is also converted into an embedding.

Development plan

Step 8 - Similarity Search

The vector database performs semantic similarity search.
Goal:
Find chunks that are most relevant to the question.

Development plan

Step 9 - Retriever Component

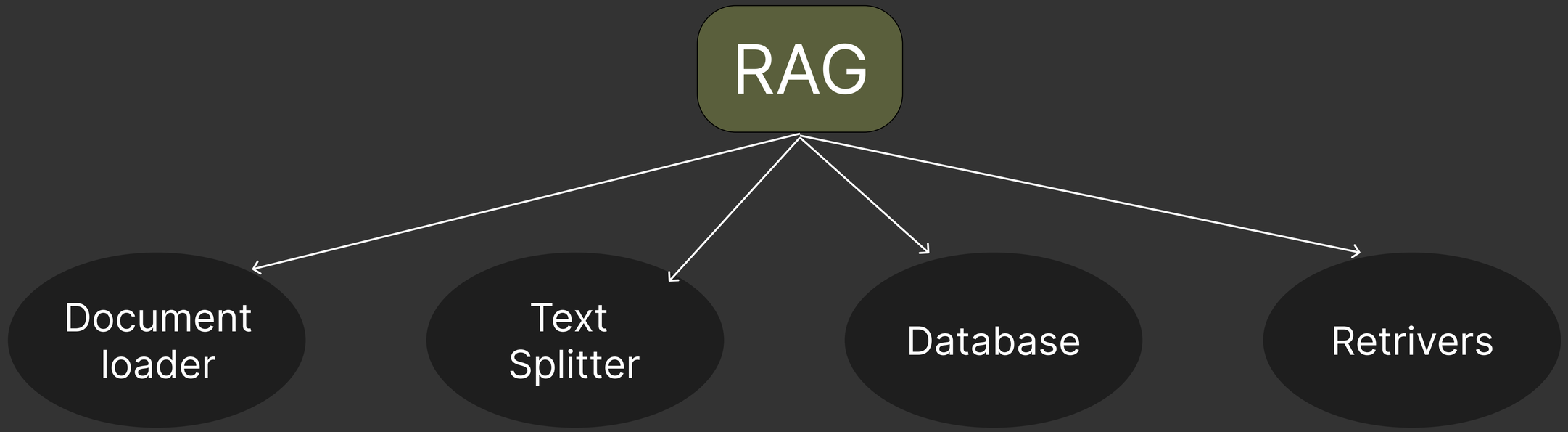
The retriever selects the top-k relevant chunks.
These chunks form the context.

Development plan

Step 10 - LLM answers

Based on the context the LLM answers.

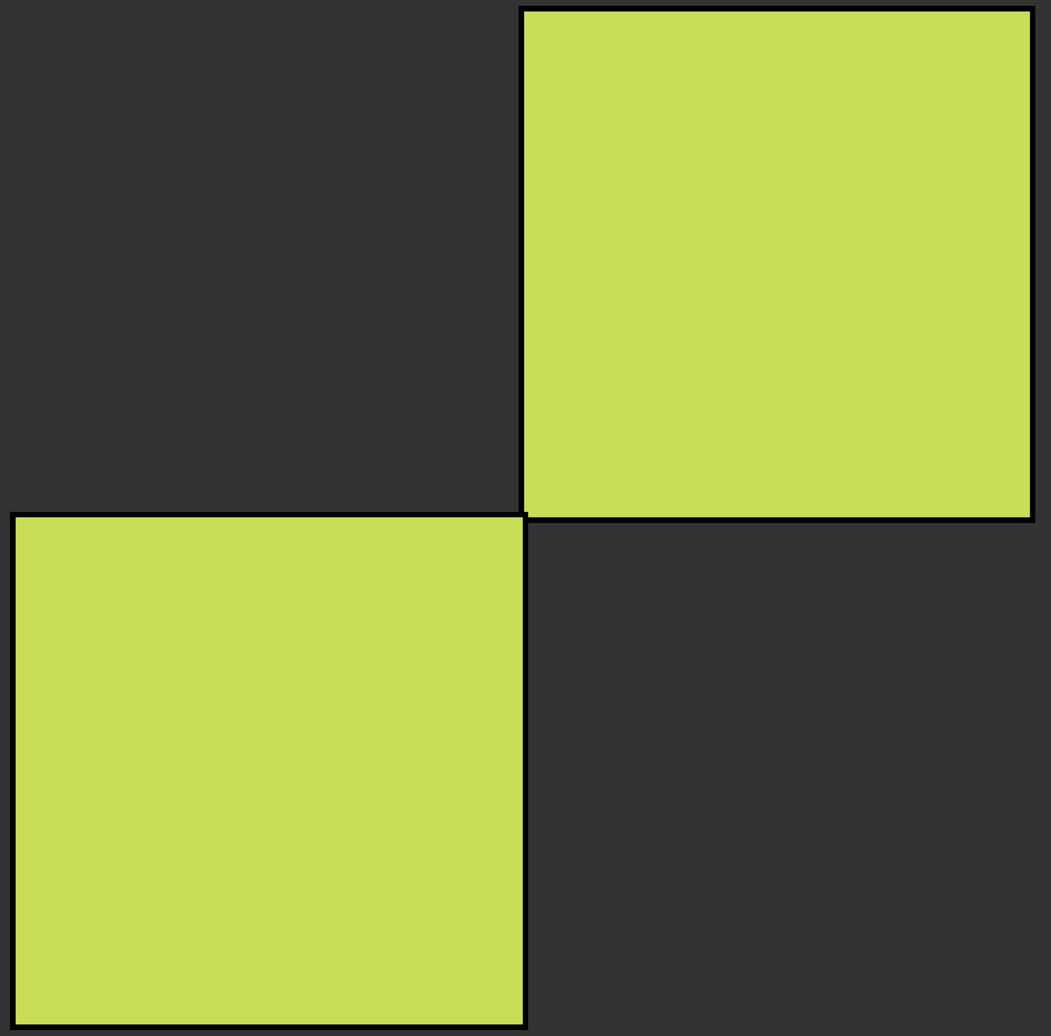
RAG



Retrieval Augmented Generation

Lets Go...

Ok so now lets create this and also I will teach each and every thing while making the project all the best.



Text Splitter

1. Most of the embedding models and language model have a context window and they cannot take infinite tokens at the same time so for that we have to use the text splitting.

Text Splitter

2. Better Retrieval in RAG

In Retrieval-Augmented Generation (RAG), we search through vector embeddings.

If the document is too large:

- embeddings become less precise

Smaller chunks allow the system to retrieve only the most relevant piece of information instead of the whole document.

Text Splitter

3. More Accurate Embeddings

Embeddings work best when the text represents one clear idea or topic.

If you embed very large text:

- multiple topics mix together
- semantic meaning becomes blurred

Chunking ensures each embedding represents a focused concept.

Text Splitter

4. Faster Processing
Working with smaller chunks:

- speeds up embedding creation
- speeds up similarity search in vector databases (Chroma, Pinecone, FAISS).

Large documents would make the pipeline slower and expensive.

Text Splitter

We are going to see 3 types of text splitting methods.

1. Character - Based Splitting
2. Token-Based Splitting
3. Semantic / Meaning-Based Splitting

Text Splitter

1. Character - Based Splitting

Character-based splitting is the simplest method of text splitting.
It divides a large document into smaller chunks based on the number of characters, without understanding words, tokens, or meaning.

Text Splitter

2. Token-Based Splitting

Token-based splitting divides a document into chunks based on the number of tokens, not characters.
Since LLMs process tokens instead of words, this method aligns better with how models actually read text.

Text Splitter

3. Semantic / Meaning-Based Splitting

Semantic / Meaning-Based Splitting divides a document into chunks based on the meaning and structure of the text, rather than characters or tokens.
The goal is to keep complete ideas together so that each chunk contains a coherent piece of information.

Text Splitter

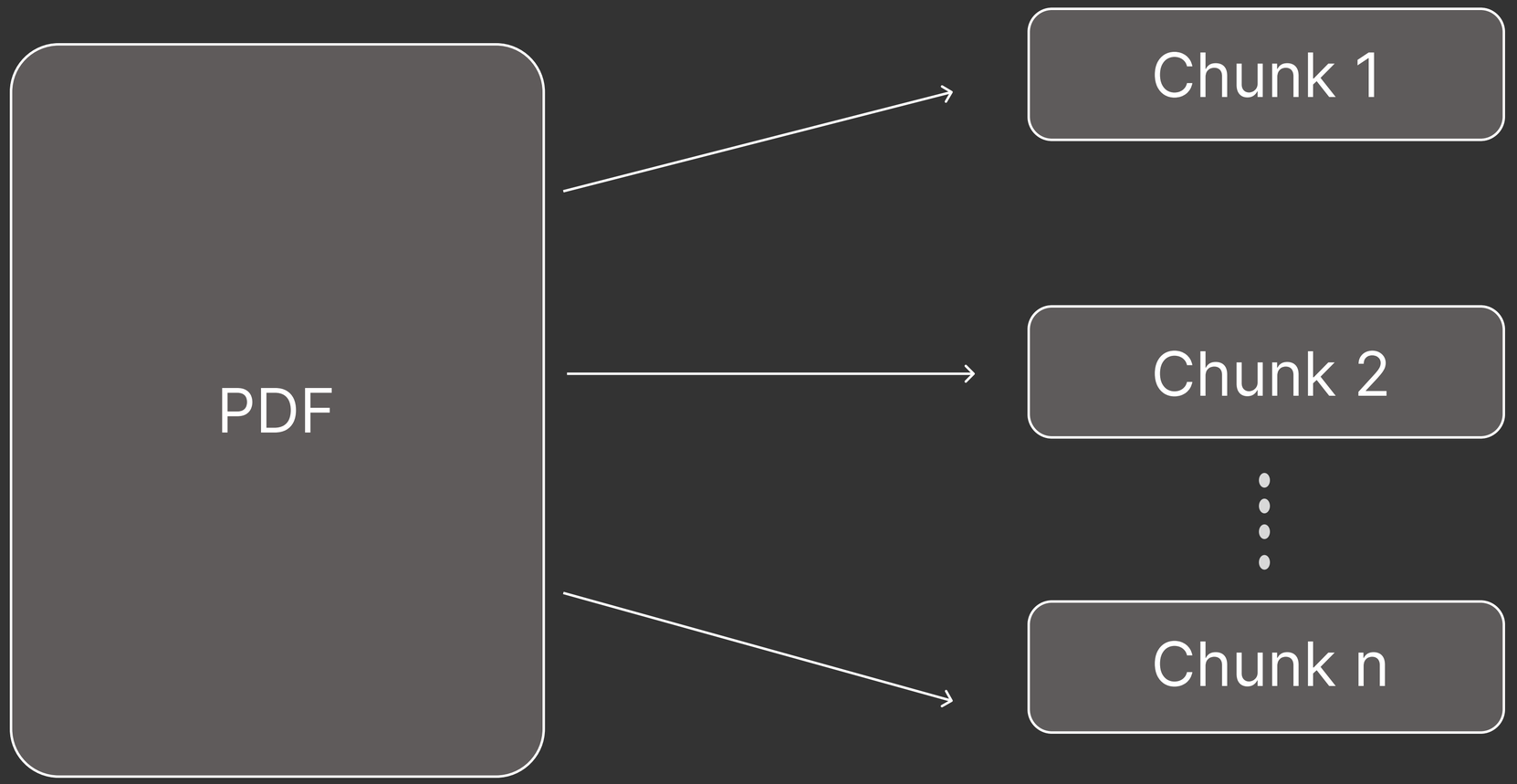
Deep learning is a specialized area of machine learning that uses neural networks with multiple layers to automatically learn complex representations from large amounts of data. Natural Language Processing (NLP) is another important area of artificial intelligence that focuses on enabling computers to understand, interpret, and generate human language in tasks such as translation, chatbots, and sentiment analysis.



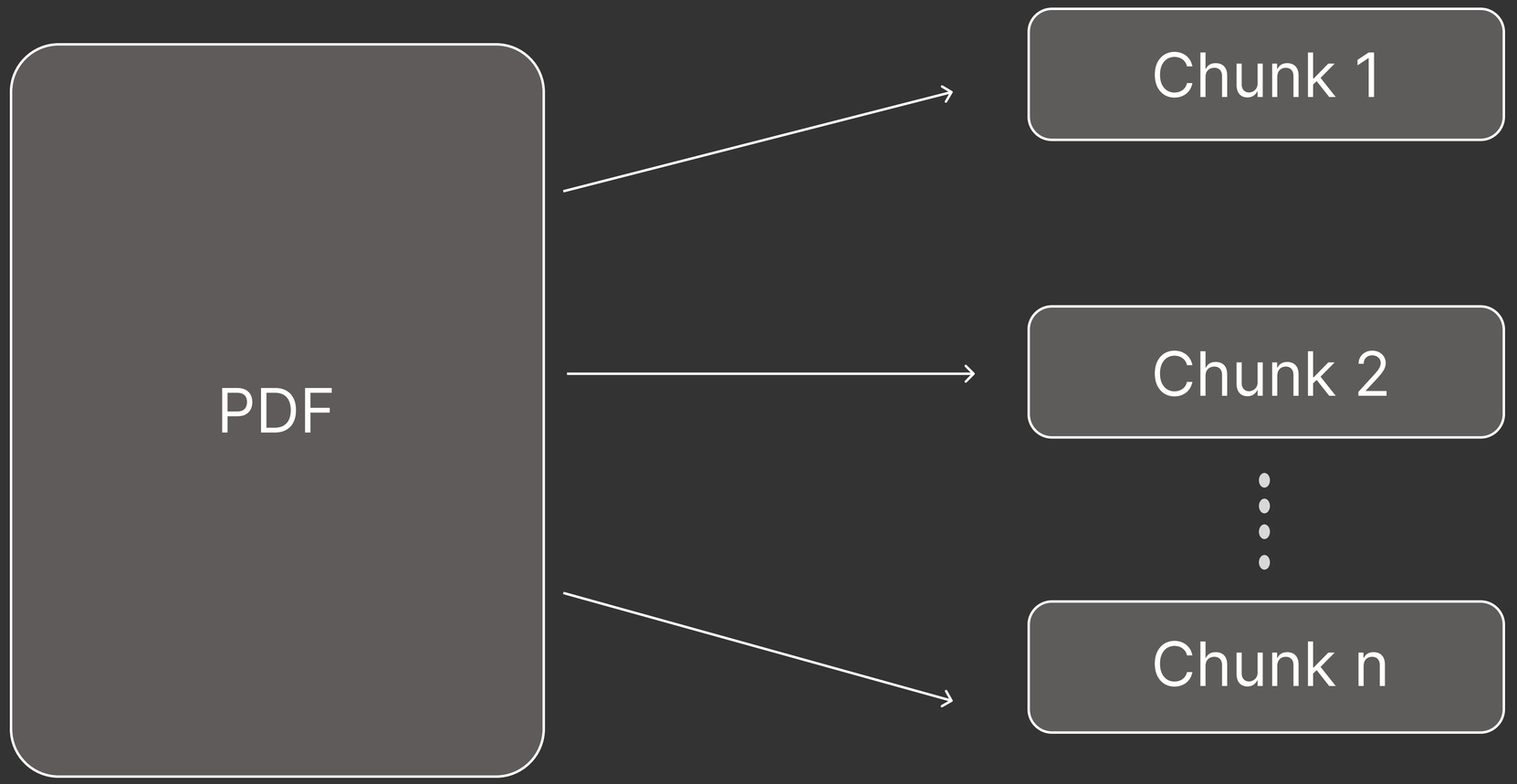
Vector Store

So first of all if we talk about our project we are creating a AI assistant which is going to search our pdf and give answers based on that.

Vector Store

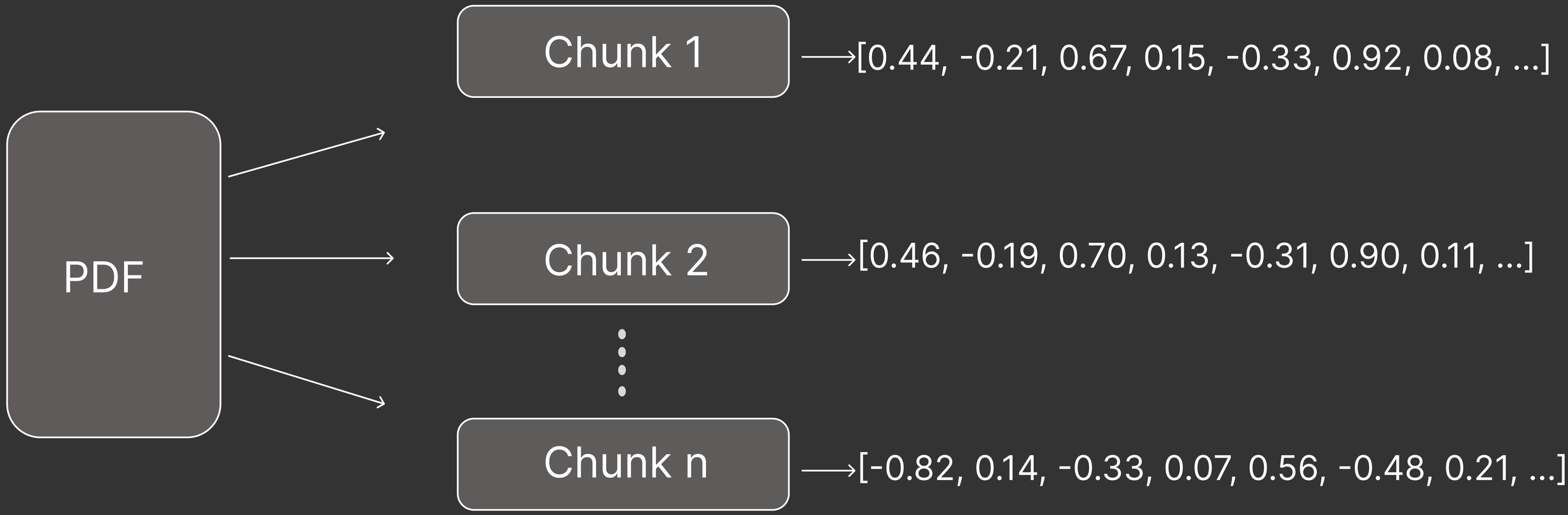


Vector Store

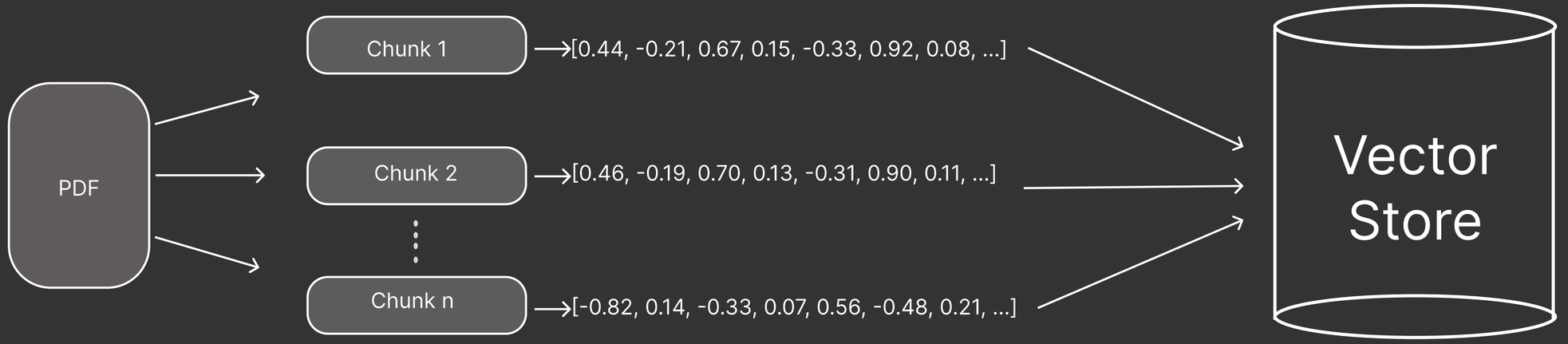


After this our next step will be to create embeddings of these chunks.

Vector Store



Vector Store



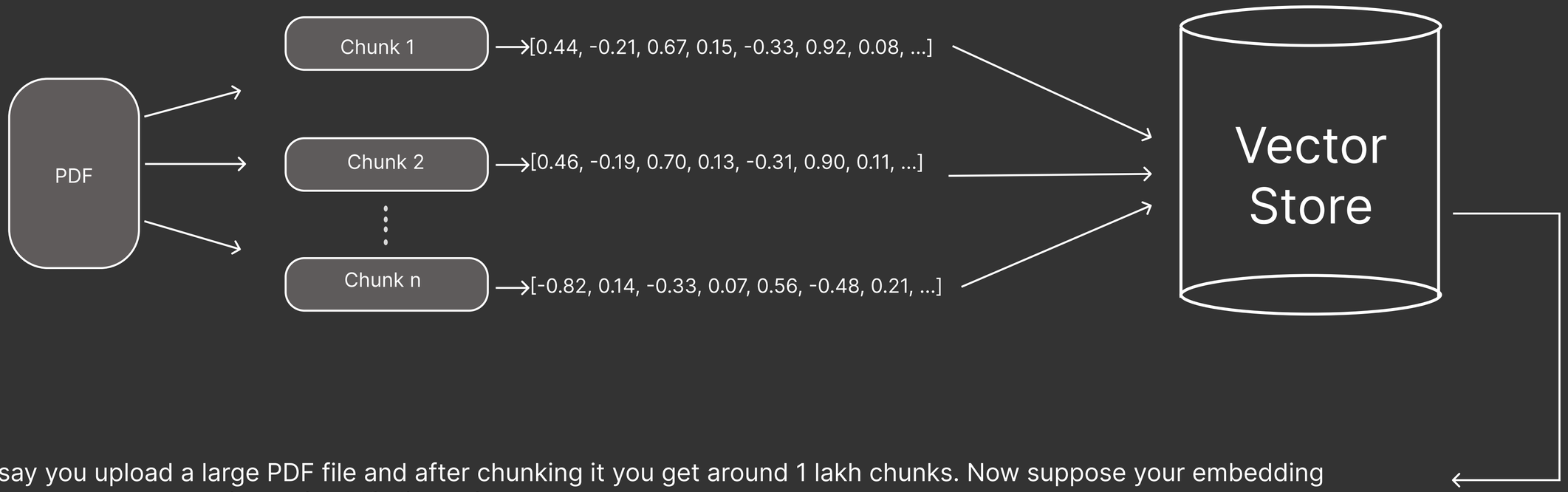
Now after this we need to store these embeddings and for that we are going to use vector store.

Vector Store

Now the question that might come to your mind is that we already have many types of databases like MySQL, PostgreSQL, MongoDB, etc. So why do we even need vector databases?

Vector Store

Answer is actually simple lets understand.



Let's say you upload a large PDF file and after chunking it you get around 1 lakh chunks. Now suppose your embedding model produces embeddings with 512 dimensions.

Vector Store



1 lakh
embedding

Now lets say this is a normal DataBase. and user give a query.

↓

What does the notes say about Natural Language processing.

Vector Store



1 lakh embedding

The Query is first converted into embedding

What does the notes say about Natural Language processing.

↓

[-0.82, 0.14, -0.33, 0.07, 0.56, -0.48, 0.21, ...]

512 - dimensions

Vector Store



1 lakh embedding

The biggest problem is this 512 dimension embedding is different from all the 1 lakh embeddings in our database so we conduct a similarity search with all the 1 lakh embeddings.

[-0.82, 0.14, -0.33, 0.07, 0.56, -0.48, 0.21, ...]

512 - dimensions

Vector Store



1 lakh embedding

so you are working at 0(n) time complexity and you are searching for 1 lakh time and then finding out and this dataset can become more big so they are not the reliable option for similarity searching.

[-0.82, 0.14, -0.33, 0.07, 0.56, -0.48, 0.21, ...]

512 - dimensions

Vector Store

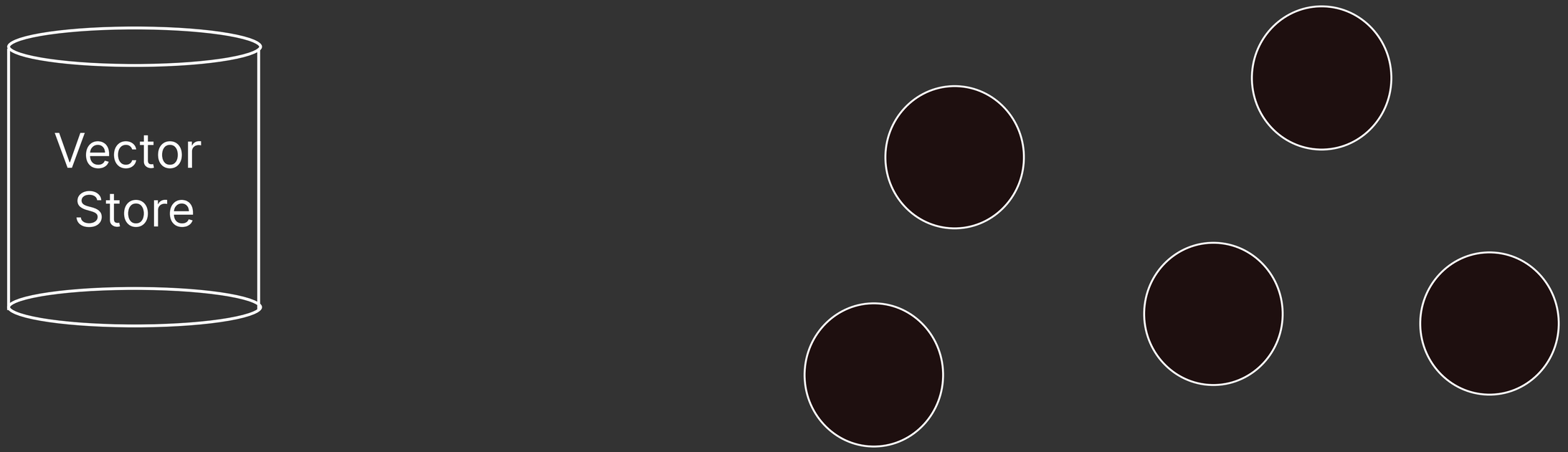


Where as vector store use Approximate Nearest Neighbour algorithms like -
HNSW
IVF
PQ

and all of these are indexing techniques
now we are not going to see all but lets see an example why they are fast.

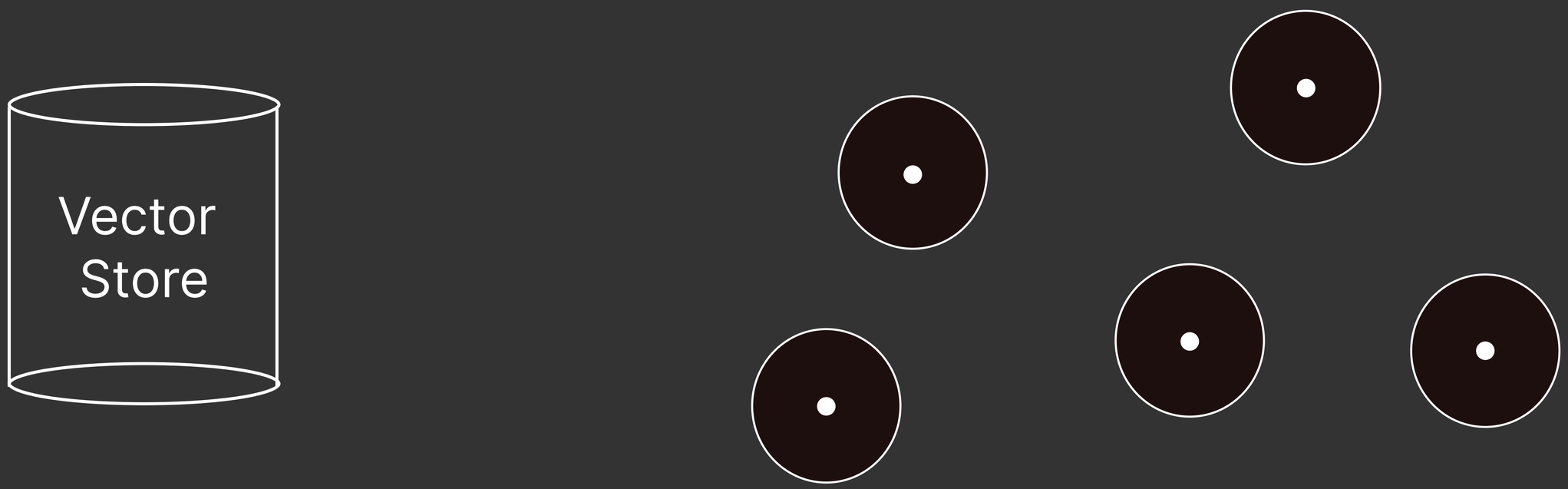
Vector Store

IVF (Inverted File Index) - Lets say we divide our Database in 5 clusters using k-means or any other algo. so each will have 20000 embeddings and all of them have some sort of similarity.



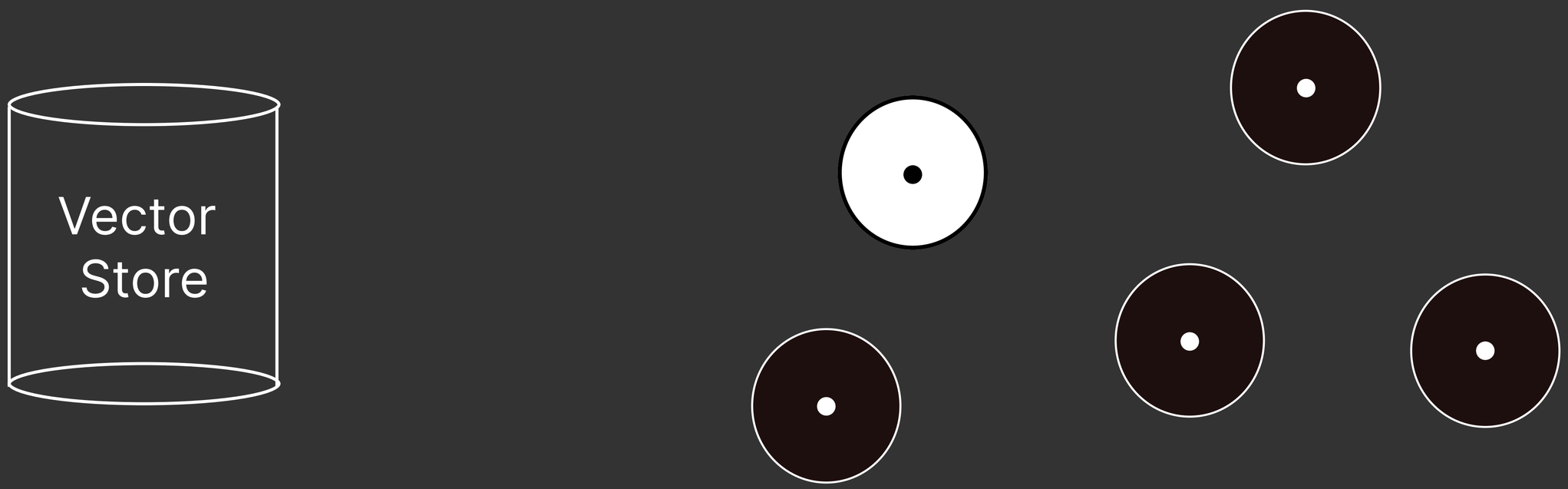
Vector Store

After that we calculate the average embedding of all the 20000 embeddings. we can call it centroid. so now we have 5 average embeddings.



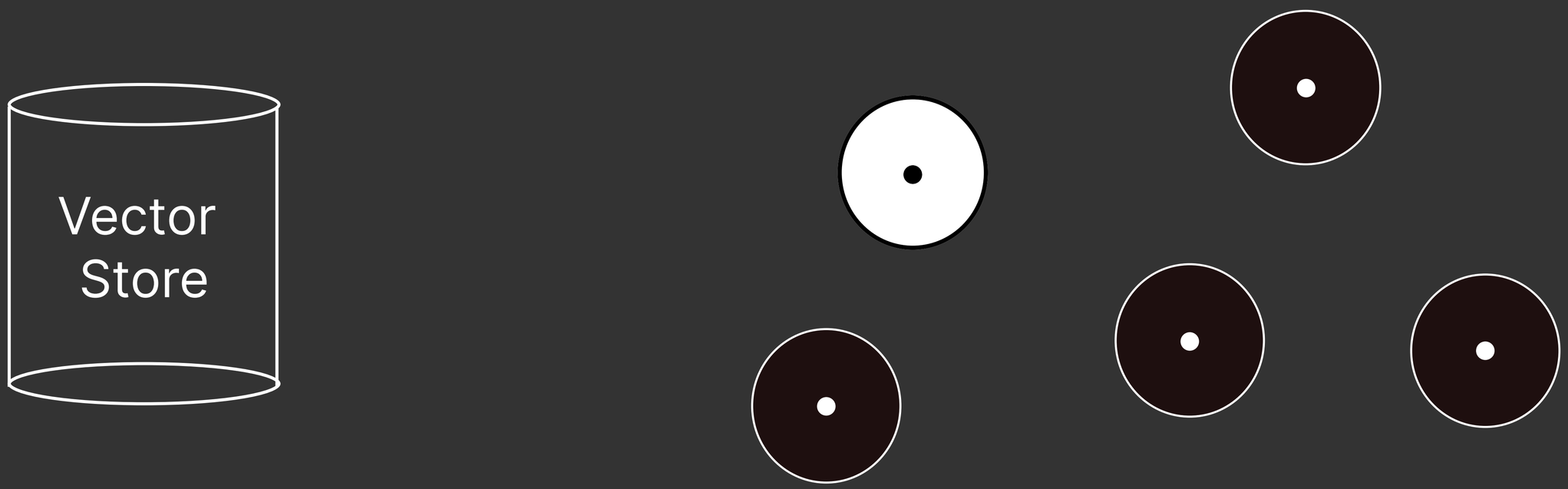
Vector Store

Now for the query embedding we can find which cluster is suitable for searching and we search in those 20000 embeddings. that is actually 5 times faster that normal database



Vector Store

Now for the query embedding we can find which cluster is suitable for searching and we search in those 20000 embeddings. that is actually 5 times faster that normal database



Vector Store

Feature	Normal Database	Vector Database
Data stored	Structured data (text, numbers, rows)	Embeddings (vectors like `[0.23, -0.41, ...]`)
Search type	Exact match	Similarity search
Query example	`WHERE movie = "Interstellar"`	"movies about space travel"
Indexing	B-Tree, Hash	HNSW, IVF, PQ
Use case	Banking, user records, transactions	AI search, recommendations, RAG
Comparison method	Equality or filtering	Cosine similarity / vector distance

Vector Store

Now there are many Vector stores like
These are widely used with frameworks like LangChain and LlamaIndex.

- Chroma
- FAISS
- Annoy

Vector Store

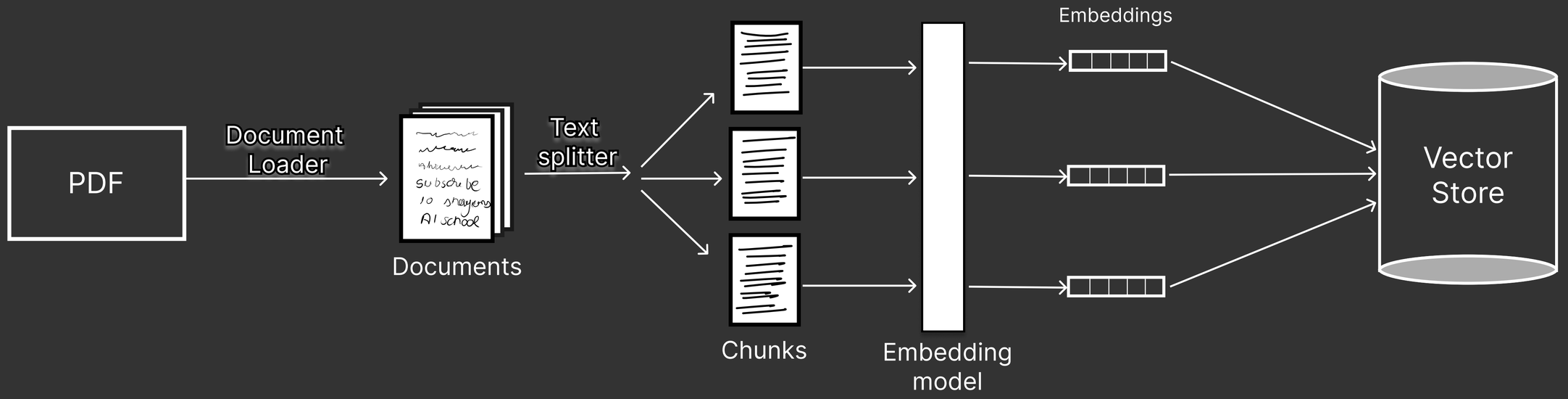
Enough Yapping lets see how to use one Vector Store.

Retrievers

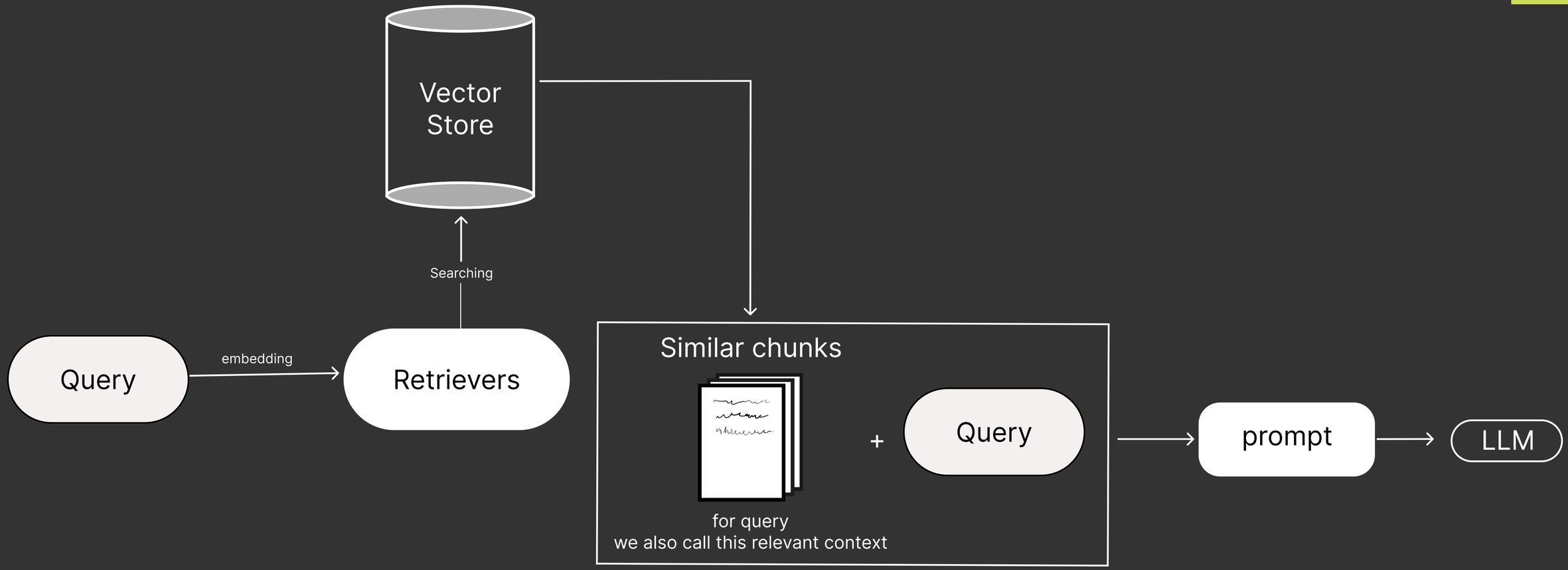
Now before understanding retrievers lets go through a diagram to understand the application that we are creating and we will also see the use of retrievers in that application.

Retrievers

Phase 1



Retrievers


Vector Store
Searching
Query
embedding
Retrievers
Similar chunks
+
Query
for query
we also call this relevant context
prompt
LLM

Retrievers

A retriever takes a user query and returns the most relevant documents or chunks from a database.

Now there are 2 types of retriever

1. by data source (Wikipedia, Arxiv, PubMed, etc.)
2. By retrieval strategy (Similarity, MMR, MultiQuery, etc.)

Retrievers

By data source

When we talk about retrievers using data sources, we mean retrievers that fetch information from specific external sources or databases instead of your own vector store. These retrievers connect to APIs or datasets and return relevant documents for the query.

Lets see an example.

Retrievers

By data source

So I hope now you get the gist how by data source works same code can generate the results from the wikipedia and other types of retrievers by data source.

Retrievers

By Retriever strategy

First go to Vector store code for an important information.

Retrievers

By Retriever strategy

First go to Vector store code for an important information.

Now you know that retrievers are not only used for basic similarity search strategies. They also provide some powerful search strategies that make them more useful, and they are runnables as well, which makes them suitable for building chains.

Retrievers

Search Strategies

In most RAG (Retrieval-Augmented Generation) systems, retrievers mainly use three core retrieval strategies. These are the ones you'll see most often in real projects and tutorials.

1. Similarity Search (Most Common)
2. MMR (Max Marginal Relevance)
3. MultiQuery Retriever

Retrievers

Search Strategies - Similarity Search

The system compares the query vector with document vectors using similarity metrics like:

- Cosine similarity (most common)
- Dot product
- Euclidean distance

The retriever finds the most similar vectors.

Top-K documents are retrieved

Retrievers

Search Strategies - MMR (maximum marginal relevance)

Lets understand this with an example :
Imagine you are building a RAG system for your students.
Your knowledge base contains many chunks about Gradient Descent.
Your database chunks look like this:

Chunk 1: Gradient descent is an optimization algorithm used in machine learning.
Chunk 2: Gradient descent minimizes the loss function.
Chunk 3: Gradient descent is an optimization that minimizes the loss function.
Chunk 4: Neural networks use gradient descent for training.
Chunk 5: Support Vector Machines are supervised learning algorithms.

Retrievers

Now the user asks
What is Gradient Descent ?

Normal Similarity search finds the most similar chunks to the query embedding.
and gives chunk 1 , chunk 2 , chunk 3

but if you see carefully all the 3 chunks are saying the same thing.

This wastes:

- context window
- token usage
- information diversity

The LLM doesn't learn new information.

Retrievers

MMR stands for:
Max Marginal Relevance

Its goal is to balance two things:
1. Relevance to the query
2. Diversity among retrieved documents

So instead of retrieving similar documents, it retrieves different but relevant documents.

Retrievers

Search Strategies - MultiQuery

Imagine you built a RAG system for machine learning notes.

Your database contains these chunks:

Chunk 1: Gradient descent is an optimization algorithm used to minimize loss functions.
Chunk 2: Neural networks are trained using optimization algorithms like gradient descent.
Chunk 3: Weight updates in machine learning models are done using gradient descent.
Chunk 4: Support Vector Machines are supervised learning algorithms.

Retrievers

Chunk 1: Gradient descent is an optimization algorithm used to minimize loss functions.
Chunk 2: Neural networks are trained using optimization algorithms like gradient descent.
Chunk 3: Weight updates in machine learning models are done using gradient descent.
Chunk 4: Support Vector Machines are supervised learning algorithms.

Now the user asks:
"What is gradient descent?"

the retriever might return:
Chunk 1
Chunk 3

Retrievers

Chunk 1: Gradient descent is an optimization algorithm used to minimize loss functions.
Chunk 2: Neural networks are trained using optimization algorithms like gradient descent.
Chunk 3: Weight updates in machine learning models are done using gradient descent.
Chunk 4: Support Vector Machines are supervised learning algorithms.

But notice something
There are other relevant chunks like Chunk 2 that might not be retrieved.
Why?
Because the query wording is limited.

Retrievers

The Core Problem
A single query might not capture all ways a concept can be expressed.
for example :
"What is gradient descent?"

could be asked as

"Explain gradient descent"
"How does gradient descent work?"
"Optimization algorithm used in neural networks"

Retrievers

The Core Problem
A single query might not capture all ways a concept can be expressed.
for example :
"What is gradient descent?"

could be asked as

"Explain gradient descent"
"How does gradient descent work?"
"Optimization algorithm used in neural networks"

Different wording → different embeddings → different search results.
This is where MultiQuery Retriever helps.

Retrievers

So Instead of using one query, it generates multiple variations of the query using an LLM. so now the pipeline becomes

User Query
↓
LLM generates multiple queries
↓
Retriever searches for each query
↓
Combine all retrieved documents

Retrievers

So now lets finally complete our project.



Made with Goodnotes

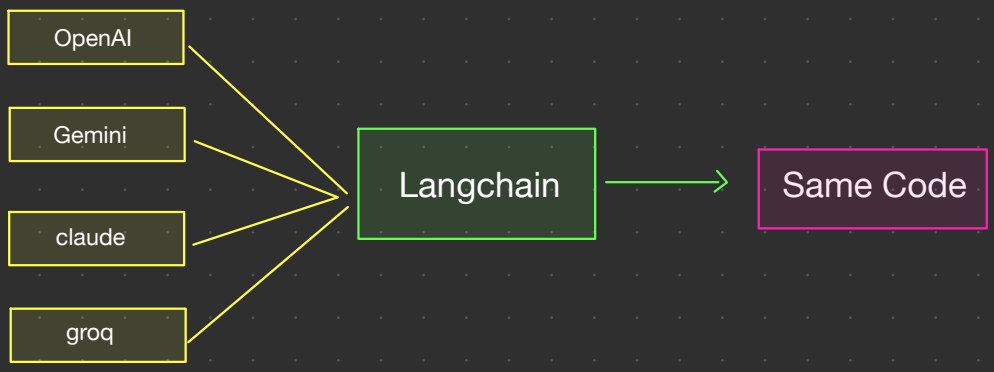
Runnables

So imagine you want to build a chatbot. For that, you obviously need an LLM.
Now earlier, there was a big problem. There were many different LLM providers like OpenAI, Anthropic, Google, and others. The issue was that each of them had their own SDK, their own syntax, and their own way of calling the model.

Because of this, if you built your project using one provider, like OpenAI, and later decided to switch to something like Gemini or Claude, you basically had to rewrite a large part of your code. That was frustrating and time-consuming.

This is where LangChain comes in and solves the problem beautifully.

LangChain provides a unified interface. That means no matter which LLM you use, the way you write your code remains almost the same. You just change the model, and everything else stays as it is.
This flexibility is one of the biggest reasons why developers love LangChain. It gives you the freedom to experiment and switch between models without rebuilding your entire project from scratch.



So once developers started using LLMs, things got interesting very quickly.

At first, it was simple. You had a model, you gave it some input, and it gave you an output. That was it.
But as people started building real applications, they realized something important a single LLM call is not enough.
Let's say you're building something like a chatbot, or a document Q&A system, or even a summarization tool. Now your application is not just one step anymore. You might need to first format the input properly, then send it to the model, and then clean or structure the output before showing it to the user.
So now instead of one step, you suddenly have multiple steps.
You have:

- • some kind of input processing
- • then a prompt template
- • then the LLM call
- • and then an output parser

At this point, you can clearly see that your application is no longer just a single model call.
Now you have multiple components involved things like LLMs, prompt templates, output parsers, and even custom logic in between.
And all of these components need to work together in a proper sequence to give you the final result.

Made with Goodnotes

Some of the most common ones are:

- • **LLM / Chat Model** → the core model that generates responses
- • **Prompt Template** → formats the input before sending it to the model
- • **Output Parser** → cleans or structures the model's response
- • **Retriever** → fetches relevant data (used in RAG systems)
- • **Document Loader** → loads data from PDFs, files, etc.
- • **Text Splitter** → breaks large text into smaller chunks
- • **Embeddings Model** → converts text into vectors
- • **Vector Store** → stores and searches embeddings

```python
from dotenv import load_dotenv
load_dotenv()

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1. Model
model = ChatOpenAI(model="gpt-4o-mini")

# 2. Prompt Template
prompt_template = ChatPromptTemplate.from_template(
    "Explain {topic} in simple words"
)

# 3. Output Parser
parser = StrOutputParser()

# Step-by-step manual flow (NO chains)

# Format the prompt
formatted_prompt = prompt_template.format_messages(topic="Machine Learning")

# Call the model (old style)
response = model.predict_messages(formatted_prompt)

# Parse the output
final_output = parser.parse(response.content)

print(final_output)
```

The base code
We and everyone
else used
to crete a basic
chatbot

So even though we now have better components like prompt templates and output parsers…
we are still manually connecting everything step by step.

So now if you look at this, even though we are using better components like prompt templates, models, and output parsers… we are still doing everything manually.
We have to:

- • format the prompt
- • call the model
- • parse the output

Now imagine you are building a large application, something like a RAG system.
In that case, you're not just dealing with one or two components anymore. You have a retriever, a prompt template, an LLM, an output parser, maybe even embeddings and a vector database.
And the problem is… you have to call each of these components manually.
First you retrieve the data, then you format the prompt, then you call the model, then you parse the output…
and you have to manage the flow between all of them yourself.
As your application grows, this quickly becomes messy and difficult to maintain.
Handling so many moving parts manually is not just repetitive, it's also error-prone.

Made with Goodnotes

```python
from dotenv import load_dotenv
load_dotenv()

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1. Model
model = ChatOpenAI(model="gpt-4o-mini")

# 2. Prompt Template
prompt_template = ChatPromptTemplate.from_template(
    "Explain {topic} in simple words"
)

# 3. Output Parser
parser = StrOutputParser()

# 🔥 Chain (everything connected automatically)
chain = prompt_template | model | parser

# Run the chain
result = chain.invoke({"topic": "Machine Learning"})

print(result)
```

So chains were definitely a big improvement.
They made our code cleaner, reduced repetition, and allowed us to connect multiple components into a single pipeline.
But as developers started building more complex applications, a new problem started to appear.
There wasn't just one type of chain.
LangChain had introduced many different chains for different use cases.
For example:

- • **LLMChain** → for simple prompt + model tasks
- • **SequentialChain** → for multi-step workflows
- • **RouterChain** → for dynamic decision making
- • **RetrievalQA Chain** → for RAG-based applications

Now on paper, this sounds useful.
But in practice, it became confuing.
As a developer, you always had to think:
"Which chain should I use for this problem?"
And sometimes, your use case didn't fit perfectly into any one chain.
So you either had to:

- • force your problem into an existing chain
- • or combine multiple chains in a complicated way

This made things harder instead of easier.
On top of that, chains were not very flexible.
If you wanted:

- • custom logic in between steps
- • parallel execution
- • or more control over how data flows

Chains started to feel restrictive.
So instead of simplifying things, the growing number of chains actually created confusion and friction for developers.
And because of all these limitations, LangChain made an important decision.
They moved away from the concept of predefined chains and introduced a much more flexible system… Runnables

Made with Goodnotes

Now this is where things become really interesting.
Instead of having different types of chains for different use cases, LangChain introduced a much simpler and more powerful idea.
They said what if every component we are using is treated the same way?
Think about it.
We already have multiple components in our application:
we have prompt templates, we have models, we have output parsers, retrievers, and even custom functions.

Now instead of treating all of these differently, LangChain introduced a unified concept…
Everything is a Runnable.
That means:
your prompt template is a runnable,
your model is a runnable
your output parser is a runnable,
even your own Python function can be a runnable.
So instead of thinking in terms of different types of chains, now you just think in terms of **runnables that can be connected together**.



Now imagine all these components as **Lego blocks**.
Each block has its own purpose. One block might be a prompt template, another might be a model, another could be an output parser, and so on.
Individually, they all do different things.
But here's the interesting part…
No matter what the block does, the way you connect one block to another is always the same.
Just like Lego.
You don't need a different way to connect different pieces. Every block follows the same connection structure.
And that's exactly what LangChain did with runnables.

Now there's one really interesting thing about runnables.
All these different components, whether it's a prompt template, a model, an output parser, or even your own function, they all expose the same kind of methods.
For example, you've already seen one of them:
👉 **invoke()**
This is used to run the component on a single input and get the output.
But that's not the only one.
You also have:
batch() when you want to process multiple inputs at once
stream() when you want to get output token by token like real-time responses

Now think about this…
Completely different components, doing completely different tasks, but all of them can be used in the exact same way.
And that is the real power here.
You don't need to learn different ways to use different components. Once you understand how one runnable works, you can use any other runnable the same way.

Made with Goodnotes

Actual Runnables

These are your components/runnables:

- • LLM
- • Prompt Template
- • Output Parser
- • Retriever
- • Tools

Basically: *they do the actual job*

Runnable Sequence

Now that we understand what runnables are, let's see how we actually use them in real applications.
We're going to learn this in three simple steps.
First, we'll start with the most basic and most important pattern a simple sequence.
In this, we just connect components like a pipeline. The output of one component becomes the input of the next. So your flow becomes something like prompt to model to parser. This is the foundation, and honestly, most applications start like this.

Parallel Runnable

Now till this point, we have only seen how to build a single pipeline where the input flows step by step and produces one output. But in real-world applications, sometimes we don't want just one result. We might want multiple outputs at the same time, like a short explanation and a detailed explanation, or maybe the same answer in different formats. Instead of running the pipeline again and again for each case, we can use a Parallel Runnable.

In this approach, we define multiple pipelines inside a dictionary, and all of them run simultaneously on the same input. Each pipeline produces its own output, and in the end, we get all the results together in a structured form. So instead of one input giving one output, now one input can give multiple outputs in a single execution, which makes our applications more powerful and efficient.

Runnable Lambda

Now sometimes in our pipeline, we don't just want to pass data forward, we might want to slightly modify it or pick a specific part of it before sending it to the next step. This is where RunnableLambda comes in. It allows us to write a simple Python function and insert it inside our pipeline as a runnable. For example, if our input is a dictionary with multiple keys, and we only want to send one specific part of it to a particular pipeline, we can use RunnableLambda to extract that part. So instead of the entire input going everywhere, we can control exactly what each component receives. In simple terms, RunnableLambda lets us add custom logic inside our runnable flow, making our pipelines more flexible and powerful.

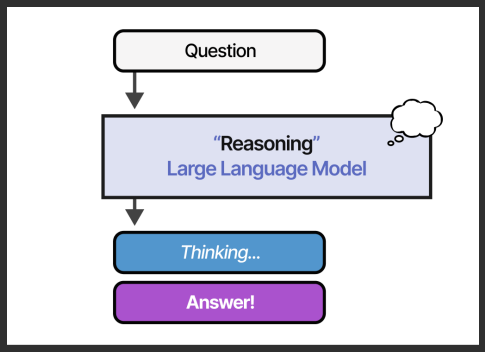
Made with Goodnotes

Runnable Passthrough

Runnable Passthrough is used when we want to keep the original input or some intermediate data while passing it through the pipeline. Normally, in a sequence, each step replaces the previous output, so earlier data gets lost. But in many real-world scenarios, we need to carry multiple pieces of information together. RunnablePassthrough allows us to forward the input as it is, without modifying it, so that it can be used along with other outputs in later steps. In simple terms, it helps us preserve data while still continuing the flow of the pipeline.

Tools

An LLM (Large Language Model) is an AI system trained to understand and generate human language.



Core capabilities an LLM can think / reasoning
And then generate an answer.

But our LLMs are not good with many things like

- No Real-Time Data
- Not Reliable for Calculations
- Cannot Call APIs
- Cannot Access Your Data

Cannot Perform Actions

Examples:

- • Send email ✗
- • Book ticket ✗
- • Run code ✗
- • Update database ✗

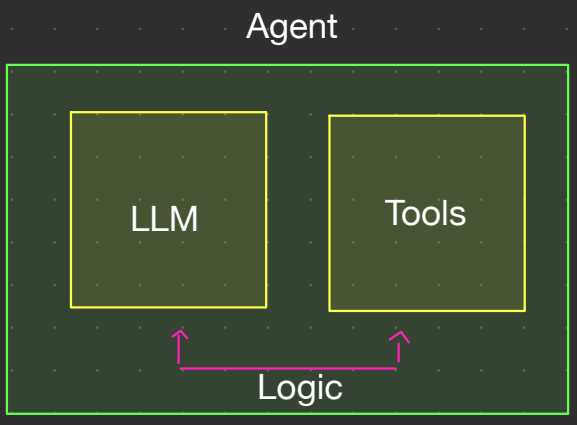
Tools are simply **functions or external services that an LLM can use to do real-world tasks**. On its own, an LLM can only generate text, but with tools it can **fetch live data, perform accurate calculations, access databases, or execute actions**. In short, tools extend an LLM from just *thinking* to actually *doing things*.

Made with Goodnotes

We use an LLM as the brain, tools to perform tasks, and our code to define the logic together, this creates an agent.

LLM decides *what to do*,
tools help *do the task*,
and our code controls *how everything works together*.

Agent = Brain (LLM) + Tools + Control Logic (our code)



Tools category

Build-in Tools

Built-in tools are tools that are already provided by a framework, so you don't have to create them from scratch. They are ready to use and help the LLM perform common tasks like searching the internet, doing calculations, or working with data. You just connect them to your LLM, and it can start using them easily.

Custom tools

Custom tools are tools that **you create yourself** based on your needs. They are usually simple functions written in code that help the LLM perform specific tasks, like accessing your database, processing your data, or calling your own APIs. This allows you to give the LLM **exact abilities that match your project**.

Made with Goodnotes

Tool Binding

Tool binding means connecting our tools to the LLM so it can use them when needed.

We give our tools to the LLM, and now the LLM knows:

- • what tools are available
- • what they do
- • when to use them

Tool calling

In tool binding, we gave tools to the LLM…
in tool calling, the LLM chooses to use them.

The LLM does NOT execute the tool it only suggests the tool call.

Tool Execution

Tool execution is when we actually run the tool after the LLM decides to use it.
LLM selects the tool → we execute it → get the result → send it back to LLM.

AI agents

So let's say today is Sunday and you want to go on a date with your girl/boy friend now what are you going to do.







You might:

- • Check weather 🌦️
- • Order food 🍔
- • Book a cab 🚗
- • Look for a movie 🎬

You are not doing one task… you are making *decisions* and using different tools.

Made with Goodnotes

Title:

Now Imagine AI Doing This
Content:
User says:
"Plan my evening"
AI should:

- • Understand intent
- • Decide steps
- • Use tools
- • Give final result

💡 This is where **Agents** come in

Agent = Brain + Tools + Decision Making

Lets Create an Agent

We are going to create a city intelligence Agent and we need some tools for that

1. Weather Tool
API: OpenWeatherMap
Purpose:

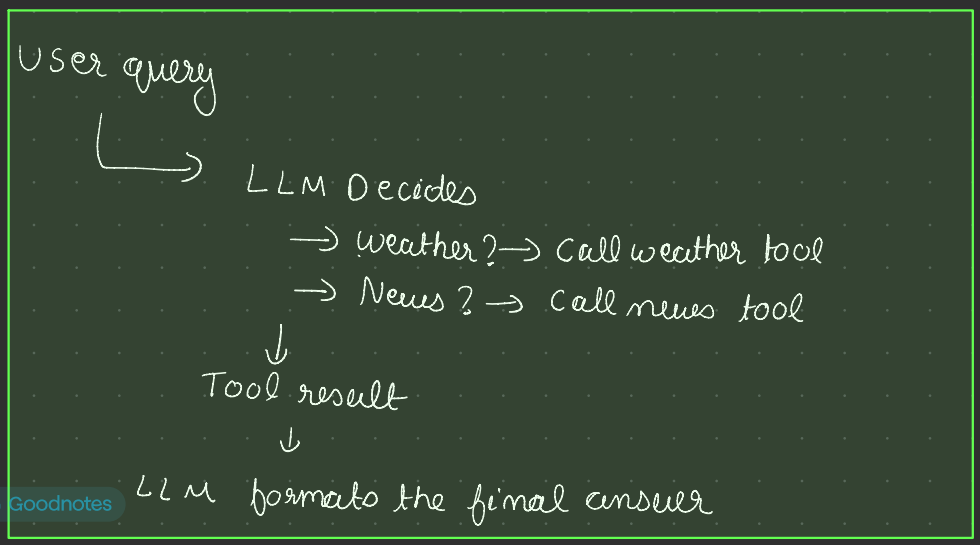
- • Get **real-time weather data**

Why it's needed:
LLM does NOT know live weather → needs external tool

2. News Tool
API: Tavily
Purpose:

- • Get **latest news about a city**

Why it's needed:
LLM does NOT know current events → needs search



Made with Goodnotes

In our code, we didn't just call an API or use an LLM separately. We created a system that could understand a user's intention, decide what action is needed, execute that action using external tools, and then use the result of that action to generate a meaningful response. For example, when the user asked about weather, the LLM first analyzed the question and decided that weather information is required. Then it triggered the weather tool, which fetched real-time data from an API. After that, we fed that result back into the LLM, and only then the LLM generated the final answer.
So what we really built was not just code we built a decision-making loop where the LLM acts like a brain and the tools act like hands interacting with the real world.

In your manual code, every single step is visible:

- • when the LLM is called
- • when a tool is triggered
- • what data is passed
- • what result comes back

You can literally trace the entire flow from user input to final output.

But there are some problems with this code I am not saying the code is bad actually its really good but lets see some problems

Too Much Boilerplate Code

In your implementation, you had to write:

- • the loop
- • tool execution logic
- • message handling
- • condition checks

All of this is **repetitive work**.

Manual Loop Management is Error-Prone

You are responsible for:

- • when to stop the loop
- • when to continue
- • handling edge cases

A small mistake like:

- • forgetting `continue`
- • misplacing a condition

👉 can break the entire agent.

Scaling Becomes Difficult

Right now you have:

- • 2 tools (weather + news)

Imagine:

- • 10 tools
- • multiple conditions
- • complex workflows

Your code will quickly become:

- • messy
- • hard to maintain
- • difficult to extend

Harder to Maintain Over Time

As your project grows:

- • more tools
- • more features
- • more logic

The manual loop becomes harder to manage.

Made with Goodnotes

`create_agent` is a high-level abstraction provided by LangChain that automates everything we manually implemented in our agent. Instead of writing the loop, handling tool calls, managing messages, and deciding when to stop, we simply define the model and the tools, and the framework takes care of the entire execution process internally. The core logic remains the same the LLM still decides which tool to use, tools still fetch external data, and results are passed back for final response generation but all of this happens behind the scenes. The main difference is that our manual approach gives full control and visibility over each step, while `create_agent` hides this complexity to make development faster, cleaner, and easier to scale.

Made with Goodnotes